

---

# Prédiction des Prix Immobiliers

en Tunisie — Application Web Django & Machine Learning

---

## Rapport de Projet

Méthodologie **CRISP-DM**

Cross-Industry Standard Process for Data Mining

### Auteurs

Mohamed Hamed Ibn Hadj Mohamed

Mahdi Oueslati

Chadi Letaief

### Établissement

École Polytechnique de Sousse

### Année académique

2025 – 2026

# Résumé

Ce rapport présente le développement complet d'un système de **prédiction des prix immobiliers en Tunisie** en suivant la méthodologie **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*). Le projet combine une application web Django avec des modèles de Machine Learning permettant d'estimer la valeur marchande de biens immobiliers sur le marché tunisien.

Le jeu de données, constitué de **5 653 annonces** collectées par scraping sur les plateformes tunisiennes Tayara, Tunisiapromo et Mubawab, couvre 12 gouvernorats et cinq catégories de biens (location d'appartements, vente d'appartements, vente de maisons, location de maisons, location de vacances).

Sur le plan de la modélisation, cinq algorithmes supervisés ont été évalués : **Régression Linéaire**, **Random Forest**, **XGBoost**, **Gradient Boosting** et **K-Plus Proches Voisins (KNN)**. Le meilleur modèle (Régression Linéaire après One-Hot Encoding sur 107 dimensions) atteint  $R^2 = 0,4708$  et un MAE de 80 337 TND sur l'ensemble de test, avec une moyenne de validation croisée de 0,8895.

L'application intègre un système d'authentification multi-rôles (Agent, Vendeur, Acheteur, Administrateur), une interface de prédiction intelligente, ainsi qu'un ensemble de fonctionnalités avancées : messagerie interne, tableaux de bord personnalisés et gestion des biens immobiliers.

**Mots-clés** : Prédiction de prix, ML, Immobilier tunisien, Django, CRISP-DM, Random Forest, XGBoost, Python.

# Table des matières

|                       |   |
|-----------------------|---|
| Résumé                | 1 |
| Introduction Générale | 6 |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Business Understanding — Compréhension Métier</b>     | <b>8</b>  |
| 1.1      | Contexte et Problématique . . . . .                      | 8         |
| 1.2      | Objectifs du Projet . . . . .                            | 8         |
| 1.2.1    | Objectif Principal . . . . .                             | 9         |
| 1.2.2    | Objectifs Secondaires . . . . .                          | 9         |
| 1.3      | Utilisateurs Cibles . . . . .                            | 9         |
| 1.4      | Critères de Succès . . . . .                             | 9         |
| <b>2</b> | <b>Data Understanding — Compréhension des Données</b>    | <b>11</b> |
| 2.1      | Sources de Données . . . . .                             | 11        |
| 2.1.1    | Plateformes Immobilières Tunisiennes . . . . .           | 11        |
| 2.2      | Description du Jeu de Données . . . . .                  | 11        |
| 2.3      | Analyse Exploratoire des Données (EDA) . . . . .         | 12        |
| 2.3.1    | Statistiques Descriptives . . . . .                      | 12        |
| 2.3.2    | Distribution par Ville (Gouvernorat) . . . . .           | 13        |
| 2.3.3    | Distribution par Catégorie de Bien . . . . .             | 13        |
| 2.3.4    | Distribution par Type de Transaction . . . . .           | 13        |
| 2.3.5    | Distribution du Nombre de Pièces . . . . .               | 14        |
| 2.4      | Qualité des Données . . . . .                            | 14        |
| <b>3</b> | <b>Data Preparation — Préparation des Données</b>        | <b>15</b> |
| 3.1      | Vue d'Ensemble du Preprocessing . . . . .                | 15        |
| 3.2      | Nettoyage des Données . . . . .                          | 16        |
| 3.2.1    | Traitement des Valeurs Aberrantes sur les Prix . . . . . | 16        |
| 3.2.2    | Extraction du Nombre de Pièces . . . . .                 | 17        |
| 3.3      | Encodage des Variables Catégorielles . . . . .           | 17        |
| 3.4      | Features Finales Retenues . . . . .                      | 18        |
| 3.5      | Séparation Train / Test . . . . .                        | 19        |
| <b>4</b> | <b>Modeling — Modélisation</b>                           | <b>20</b> |
| 4.1      | Choix des Algorithmes . . . . .                          | 20        |
| 4.2      | Algorithmes Utilisés . . . . .                           | 20        |
| 4.2.1    | Régression Linéaire (Baseline) . . . . .                 | 20        |
| 4.2.2    | Random Forest . . . . .                                  | 20        |
| 4.2.3    | Gradient Boosting . . . . .                              | 21        |
| 4.2.4    | XGBoost . . . . .                                        | 22        |
| 4.2.5    | K-Plus Proches Voisins (KNN) . . . . .                   | 22        |
| 4.3      | Validation Croisée . . . . .                             | 22        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 4.4      | Sauvegarde des Modèles . . . . .                           | 23        |
| <b>5</b> | <b>Evaluation — Évaluation</b>                             | <b>24</b> |
| 5.1      | Métriques d'Évaluation . . . . .                           | 24        |
| 5.1.1    | Définitions . . . . .                                      | 24        |
| 5.2      | Résultats Comparatifs des Modèles . . . . .                | 25        |
| 5.3      | Analyse du Meilleur Modèle : Régression Linéaire . . . . . | 26        |
| 5.4      | Discussion : Écart avec la Littérature . . . . .           | 27        |
| 5.5      | Pertinence Métier . . . . .                                | 27        |
| <b>6</b> | <b>Deployment — Déploiement</b>                            | <b>28</b> |
| 6.1      | Architecture de l'Application Web . . . . .                | 28        |
| 6.1.1    | Architecture MVT Django . . . . .                          | 28        |
| 6.1.2    | Stack Technologique . . . . .                              | 29        |
| 6.2      | Fonctionnalités de la Plateforme Web . . . . .             | 29        |
| 6.2.1    | Page d'Accueil . . . . .                                   | 30        |
| 6.2.2    | Système d'Authentification Multi-Rôles . . . . .           | 31        |
| 6.2.3    | Gestion des Biens Immobiliers . . . . .                    | 32        |
| 6.2.4    | Module de Prédiction . . . . .                             | 35        |
| 6.2.5    | Fonctionnalités Avancées . . . . .                         | 37        |
| 6.3      | API REST de Prédiction . . . . .                           | 38        |
| 6.4      | Infrastructure de Déploiement Production . . . . .         | 39        |
| 6.4.1    | Pipeline CI/CD . . . . .                                   | 39        |
| 6.4.2    | Sécurité . . . . .                                         | 39        |
|          | <b>Conclusion et Perspectives</b>                          | <b>41</b> |
|          | <b>Références Bibliographiques</b>                         | <b>43</b> |
| <b>A</b> | <b>Annexe A : Structure du Projet</b>                      | <b>45</b> |
| <b>B</b> | <b>Annexe B : Installation et Démarrage Rapide</b>         | <b>47</b> |
| <b>C</b> | <b>Annexe C : Métriques Détaillées par Modèle</b>          | <b>49</b> |

# Table des figures

|      |                                                                           |    |
|------|---------------------------------------------------------------------------|----|
| 1    | Le cycle CRISP-DM appliqué à la prédiction des prix immobiliers tunisiens | 7  |
| 1.1  | Pipeline méthodologique global du projet . . . . .                        | 10 |
| 5.1  | Comparaison des modèles basée sur le $R^2$ (ensemble de test) . . . . .   | 26 |
| 5.2  | Comparaison des modèles basée sur le RMSE (ensemble de test) . . . . .    | 26 |
| 6.1  | Page d'accueil de l'application web . . . . .                             | 30 |
| 6.2  | Page d'inscription avec choix de rôle . . . . .                           | 31 |
| 6.3  | Page de connexion sécurisée . . . . .                                     | 31 |
| 6.4  | Liste des biens immobiliers disponibles . . . . .                         | 32 |
| 6.5  | Détail d'un bien immobilier (villa) . . . . .                             | 33 |
| 6.6  | Formulaire d'ajout d'un bien immobilier . . . . .                         | 34 |
| 6.7  | Formulaire de prédiction de prix immobilier . . . . .                     | 35 |
| 6.8  | Résultat d'estimation avec score de confiance . . . . .                   | 36 |
| 6.9  | Interface de recherche avancée avec filtres . . . . .                     | 37 |
| 6.10 | Tableau de bord statistiques du marché immobilier tunisien . . . . .      | 38 |

# Liste des tableaux

|     |                                                                                         |    |
|-----|-----------------------------------------------------------------------------------------|----|
| 1.1 | Profils utilisateurs et besoins . . . . .                                               | 9  |
| 2.1 | Variables du dataset — Description des attributs . . . . .                              | 12 |
| 2.2 | Statistiques descriptives de la variable cible <code>prix_num</code> (en TND) . . . . . | 12 |
| 2.3 | Répartition des annonces par gouvernorat . . . . .                                      | 13 |
| 2.4 | Répartition des annonces par catégorie . . . . .                                        | 13 |
| 2.5 | Répartition par nombre de pièces (notation tunisienne S+n) . . . . .                    | 14 |
| 2.6 | Rapport de qualité des données . . . . .                                                | 14 |

|     |                                                                                          |    |
|-----|------------------------------------------------------------------------------------------|----|
| 3.1 | Récapitulatif du preprocessing : avant et après . . . . .                                | 16 |
| 3.2 | Stratégies d'encodage des variables catégorielles . . . . .                              | 17 |
| 3.3 | Features sélectionnées pour la modélisation . . . . .                                    | 18 |
| 5.1 | Comparaison des performances des modèles sur l'ensemble de test (hold-out 20%) . . . . . | 25 |
| 6.1 | Technologies utilisées dans le projet . . . . .                                          | 29 |
| B.1 | Scripts de maintenance disponibles . . . . .                                             | 48 |
| C.1 | Résultats complets de la validation croisée (5-fold) . . . . .                           | 49 |
| C.2 | Hyperparamètres optimaux retenus . . . . .                                               | 49 |

# Introduction Générale

Le secteur immobilier est l'un des piliers de l'économie tunisienne, reflétant directement le pouvoir d'achat et la situation financière de la population. Selon la Banque mondiale, la population urbaine de la Tunisie a dépassé **70 %** en 2022, accentuant la pression sur le marché du logement et soulignant le besoin de mécanismes de tarification fiables.

L'estimation de la valeur d'un bien immobilier repose traditionnellement sur l'expertise humaine, souvent subjective et chronophage. Cette ambiguïté crée un écart persistant entre les prix demandés et les conditions réelles du marché, entraînant des biens invendus pendant de longues périodes, des pertes financières pour les vendeurs et une incertitude pour les acheteurs. L'essor du Machine Learning offre aujourd'hui une opportunité unique d'automatiser et d'objectiver ce processus.

Ce projet s'inscrit dans cette démarche en proposant une solution intégrée alliant une **application web Django** et des **modèles d'apprentissage automatique**. L'objectif est double :

- Fournir aux professionnels de l'immobilier (agents, vendeurs, investisseurs) un outil d'aide à la décision fiable et accessible, spécifiquement adapté au marché tunisien.
- Démontrer l'application concrète de la méthodologie CRISP-DM sur un problème réel à forte valeur ajoutée, en utilisant des données collectées sur les plateformes tunisiennes Tayara, Tunisiapromo et Mubawab.

Le rapport est organisé selon les **six phases de CRISP-DM**, permettant une traçabilité complète du raisonnement, des choix techniques et des résultats obtenus.

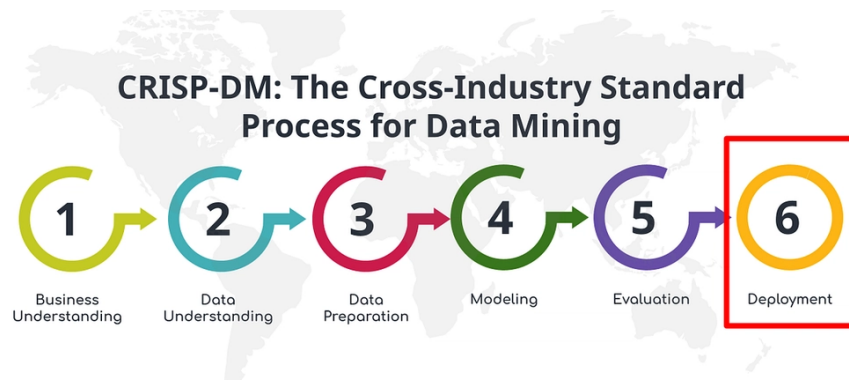


FIGURE 1 – Le cycle CRISP-DM appliqué à la prédiction des prix immobiliers tunisiens



# Chapitre 1

## Business Understanding — Compréhension Métier

**Objectif CRISP-DM :** Définir clairement le problème métier, les objectifs du projet et les critères de succès avant toute manipulation de données.

### 1.1 Contexte et Problématique

Le secteur immobilier tunisien génère des transactions considérables chaque année. L'estimation du prix d'un bien est une étape critique pour toutes les parties prenantes. Or, l'évaluation immobilière en Tunisie reste rigide, s'appuyant largement sur des estimations subjectives et biaisées fournies par les propriétaires ou les agents immobiliers, faute d'une méthodologie standardisée et fondée sur les données. Les méthodes traditionnelles présentent plusieurs limites :

- **Subjectivité** : les estimations varient d'un expert à l'autre, sans référence objective au marché réel.
- **Temps** : une expertise immobilière peut prendre plusieurs jours.
- **Coût** : le recours à des professionnels est onéreux.
- **Accessibilité** : les particuliers n'ont pas accès aux mêmes outils que les professionnels.
- **Transparence limitée** : contrairement aux marchés européens ou asiatiques, le marché tunisien n dispose pas de registre de transactions publiquement accessible.

### 1.2 Objectifs du Projet

### 1.2.1 Objectif Principal

Développer un système de prédiction automatique des prix immobiliers tunisiens capable de fournir une estimation fiable, rapide et accessible à tous les acteurs du marché, en exploitant les données des plateformes en ligne Tayara, Tunisiapromo et Mubawab.

### 1.2.2 Objectifs Secondaires

1. Construire un jeu de données représentatif du marché immobilier tunisien par scraping web des principales plateformes d'annonces.
2. Comparer systématiquement cinq algorithmes de Machine Learning sous des conditions expérimentales identiques.
3. Analyser l'importance des variables pour révéler les facteurs clés influençant les prix dans le contexte tunisien.
4. Créer une plateforme web intuitive avec gestion des rôles accessible aux non-experts.
5. Intégrer des modèles ML performants dans un environnement de production.

## 1.3 Utilisateurs Cibles

TABLE 1.1 – Profils utilisateurs et besoins

| Rôle                    | Besoin principal                        | Décision aidée                           |
|-------------------------|-----------------------------------------|------------------------------------------|
| <b>Agent immobilier</b> | Conseiller les clients avec objectivité | Fixer un prix de vente cohérent          |
| <b>Vendeur</b>          | Évaluer son bien rapidement             | Choisir le prix de mise en vente         |
| <b>Acheteur</b>         | Vérifier la justesse d'un prix          | Décider d'acheter ou de négocier         |
| <b>Investisseur</b>     | Identifier des opportunités             | Évaluer la rentabilité d'un bien         |
| <b>Banque</b>           | Valoriser un actif pour crédit          | Accorder ou refuser un prêt hypothécaire |

## 1.4 Critères de Succès

### Critères techniques :

- $R^2 \geq 0,45$  sur l'ensemble de test (contrainte spécifique au marché tunisien mixte location/vente)

- $MAE \leq 85\,000$  TND
- Temps de réponse de l'API  $< 2$  secondes
- Disponibilité de la plateforme  $> 99,9\%$

**Critères métier :**

- Adoption par les 3 types d'utilisateurs cibles (Agent, Vendeur, Acheteur)
- Taux de satisfaction utilisateur  $\geq 80\%$
- Premier outil de prédiction de prix adapté spécifiquement au marché tunisien

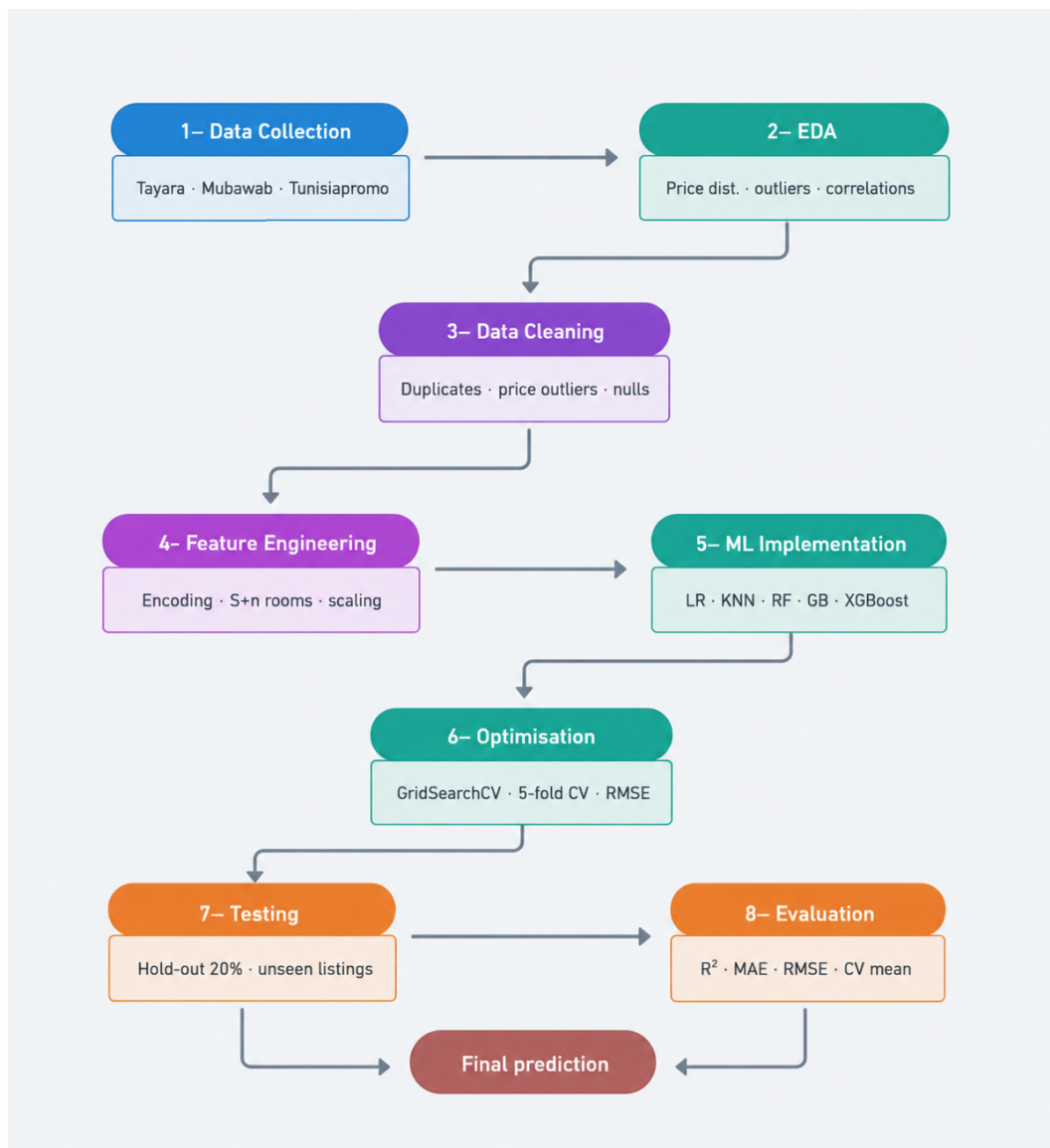


FIGURE 1.1 – Pipeline méthodologique global du projet

# Chapitre 2

## Data Understanding — Compréhension des Données

**Objectif CRISP-DM :** Collecter les données initiales, explorer leur structure et leur qualité, et identifier les premières tendances.

### 2.1 Sources de Données

#### 2.1.1 Plateformes Immobilières Tunisiennes

Le jeu de données a été constitué par scraping des annonces immobilières disponibles en ligne sur trois plateformes tunisiennes principales :

- **Tayara** ([tayara.tn](https://tayara.tn)) : principale plateforme d'annonces en Tunisie, couvrant toutes les régions.
- **Tunisiapromo** ([tunisiapromo.com](https://tunisiapromo.com)) : portail spécialisé en immobilier neuf et ancien.
- **Mubawab** ([mubawab.tn](https://mubawab.tn)) : plateforme régionale active sur le marché maghrébin.

Après déduplication et filtrage qualité, le jeu de données final contient **5 653 annonces validées**.

### 2.2 Description du Jeu de Données

Le dataset principal est `real_estate_data.csv`, contenant les annonces immobilières tunisiennes avec leurs caractéristiques et prix associés.

TABLE 2.1 – Variables du dataset — Description des attributs

| Variable                  | Type                   | Description                                           |
|---------------------------|------------------------|-------------------------------------------------------|
| <code>titre</code>        | Texte                  | Titre de l’annonce (ex : «Villa à vendre S+3»)        |
| <code>prix</code>         | Texte                  | Prix affiché en TND (chaîne brute, ex : «650 DT»)     |
| <code>prix_num</code>     | Numérique (cible)      | Prix numérique en TND                                 |
| <code>categorie</code>    | Catégorielle           | Type de bien et transaction combinés (5 catégories)   |
| <code>ville</code>        | Catégorielle           | Gouvernorat (ex : Monastir, Sfax, Sousse, Ariana...)  |
| <code>localisation</code> | Catégorielle           | Quartier ou district (ex : Bekalta, Manouba...)       |
| <code>type</code>         | Catégorielle           | Type de transaction : <i>vente</i> ou <i>location</i> |
| <code>nombre_p</code>     | Catégorielle (discret) | Nombre de pièces (notation S+n : S+0, S+1...S+11)     |
| <code>date_posted</code>  | Date                   | Date de publication de l’annonce                      |
| <code>month_posted</code> | Numérique              | Mois de publication (1–12)                            |
| <code>year_posted</code>  | Numérique              | Année de publication (2025 ou 2026)                   |

## 2.3 Analyse Exploratoire des Données (EDA)

### 2.3.1 Statistiques Descriptives

L’analyse exploratoire révèle une distribution de prix fortement asymétrique à droite, caractéristique de la coexistence d’annonces de location (prix bas) et de vente (prix élevés) dans un même dataset.

TABLE 2.2 – Statistiques descriptives de la variable cible `prix_num` (en TND)

| Variable                    | Min | Max         | Moyenne | Médiane | Éc.-type  |
|-----------------------------|-----|-------------|---------|---------|-----------|
| <code>prix_num</code> (TND) | 0   | 210 000 000 | 260 278 | 3 500   | 3 889 321 |

### 2.3.2 Distribution par Ville (Gouvernorat)

TABLE 2.3 – Répartition des annonces par gouvernorat

| Gouvernorat  | Nombre d'annonces | Pourcentage |
|--------------|-------------------|-------------|
| Nabeul       | 1 832             | 32,4%       |
| Tunis        | 1 671             | 29,6%       |
| Ariana       | 1 136             | 20,1%       |
| Sousse       | 797               | 14,1%       |
| Mahdia       | 103               | 1,8%        |
| Ben Arous    | 62                | 1,1%        |
| Autres       | 52                | 0,9%        |
| <b>Total</b> | <b>5 653</b>      | <b>100%</b> |

### 2.3.3 Distribution par Catégorie de Bien

TABLE 2.4 – Répartition des annonces par catégorie

| Catégorie             | Nombre       | Pourcentage |
|-----------------------|--------------|-------------|
| Location Appartements | 2 341        | 41,4%       |
| Vente Appartements    | 1 431        | 25,3%       |
| Vente Maisons         | 975          | 17,3%       |
| Location Maisons      | 697          | 12,3%       |
| Location de vacances  | 209          | 3,7%        |
| <b>Total</b>          | <b>5 653</b> | <b>100%</b> |

### 2.3.4 Distribution par Type de Transaction

Le dataset présente une légère majorité de biens en **location** (3 247 annonces, 57,4%) par rapport à la **vente** (2 406 annonces, 42,6%). Cette bimodalité des prix entre location et vente constitue la principale difficulté de modélisation.

### 2.3.5 Distribution du Nombre de Pièces

TABLE 2.5 – Répartition par nombre de pièces (notation tunisienne S+n)

| Nombre de pièces | Annonces | %     |
|------------------|----------|-------|
| S+2              | 2 041    | 36,1% |
| S+3              | 1 373    | 24,3% |
| S+1              | 1 142    | 20,2% |
| S+4              | 583      | 10,3% |
| S+0              | 249      | 4,4%  |
| S+5              | 228      | 4,0%  |
| S+6 et plus      | 37       | 0,7%  |

## 2.4 Qualité des Données

TABLE 2.6 – Rapport de qualité des données

| Variable         | Valeurs manquantes | %    | Action                                                        |
|------------------|--------------------|------|---------------------------------------------------------------|
| prix_num (cible) | 25                 | 0,4% | Détection et écrêtage des outliers (0 TND, max 2 000 000 TND) |
| categorie        | 0                  | 0%   | Encodage label + one-hot                                      |
| ville            | 0                  | 0%   | Encodage label + one-hot                                      |
| localisation     | 0                  | 0%   | Encodage label + one-hot                                      |
| nombre_p         | 0                  | 0%   | Extraction regex depuis notation S+n                          |

**Contrainte majeure :** Aucune colonne de surface (m<sup>2</sup>) n'est disponible dans les données scrapées, car cette variable ne pouvait pas être extraite de manière fiable depuis les annonces. Cette absence constitue le principal facteur limitant la précision des prédictions.

# Chapitre 3

## Data Preparation — Préparation des Données

**Objectif CRISP-DM :** Construire le dataset final prêt à l'entraînement. C'est l'étape la plus chronophage, représentant 60–80% du temps projet.

### 3.1 Vue d'Ensemble du Preprocessing

Le dataset initial de 5 653 enregistrements et 12 colonnes brutes a été réduit à **8 variables utiles** (7 features + 1 variable cible) après nettoyage. Aucune imputation n'a été nécessaire, le taux de rétention des enregistrements étant de **100%**.



TABLE 3.1 – Récapitulatif du preprocessing : avant et après

| Étape                          | Problème identifié (avant)                                                                                                                                                                                                             | Action réalisée (après)                                                                                                                                      |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Suppression de colonnes        | Colonnes inutiles à la modélisation : <code>titre</code> (texte libre), <code>prix</code> (chaîne brute), <code>time_posted</code> , <code>date_posted</code> (redondants avec <code>month_posted</code> et <code>year_posted</code> ) | Suppression de 4 colonnes ; dataset réduit de 12 à 8 colonnes utiles ( <code>prix_num</code> conservé comme variable cible <code>price</code> )              |
| Nettoyage des prix             | <code>prix_num</code> contient des outliers : min 0 TND, max 210 000 000 TND, médiane 3 500 TND vs moyenne 260 278 TND (forte asymétrie)                                                                                               | Détection et écrêtage des annonces à prix nul et à prix extrêmes ; distribution réelle préservée et limitation documentée                                    |
| Encodage <code>nombre_p</code> | Nombre de pièces en notation tunisienne (S+0, S+1, S+2...S+11) — non directement numérique                                                                                                                                             | Entier extrait après «S+» via expression régulière ; colonne convertie en variable entière <code>rooms</code>                                                |
| Encodage catégoriel            | Variables <code>categorie</code> (5 catégories), <code>ville</code> (10 gouvernorats), <code>localisation</code> (nombreux districts), <code>type</code> (vente/-location) non directement utilisables                                 | Label Encoding appliqué pour tous les modèles ; One-Hot Encoding supplémentaire pour Régression Linéaire et KNN : dimensions (5 653, 107) après OHE          |
| Normalisation des features     | Variables numériques ( <code>rooms</code> , <code>month_posted</code> , <code>year_posted</code> ) sur des échelles différentes, affectant les algorithmes basés sur la distance                                                       | StandardScaler appliqué pour KNN et Régression Linéaire ; aucune normalisation pour les modèles arborescents (RF, GB, XGBoost)                               |
| Séparation du dataset          | Un seul split hold-out peut produire des estimations à forte variance ; pas de surface disponible comme feature supplémentaire                                                                                                         | Split stratifié 80/20 train-test ; validation croisée à 5 folds sur l'ensemble d'entraînement ; pipeline ajusté exclusivement sur les données d'entraînement |

## 3.2 Nettoyage des Données

### 3.2.1 Traitement des Valeurs Aberrantes sur les Prix

```

1 import pandas as pd
2
3 df = pd.read_csv("real_estate_data.csv")
4
5 # Suppression des prix nuls

```

```

6 df = df[df['prix_num'] > 0]
7
8 # Statistiques avant   crtage
9 print(f"Min : {df['prix_num'].min():,} TND")
10 print(f"Max : {df['prix_num'].max():,} TND")
11 print(f"M diane : {df['prix_num'].median():,} TND")
12 print(f"Moyenne : {df['prix_num'].mean():,.0f} TND")
13
14 # Ecr tage des prix extr mes (>210 000 000 TND)
15 upper_cap = 210_000_000
16 df = df[df['prix_num'] <= upper_cap]
17 print(f"Lignes conserv es : {len(df)}")

```

Listing 3.1 – Détection et écrêtage des outliers sur prix\_num

### 3.2.2 Extraction du Nombre de Pièces

```

1 import re
2
3 def extract_rooms(s):
4     """Extrait le nombre entier depuis la notation tunisienne S+
5         n."""
6     match = re.search(r'S\+(\d+)', str(s), re.IGNORECASE)
7     return int(match.group(1)) if match else 0
8
9 df['rooms'] = df['nombre_p'].apply(extract_rooms)
10 print(df['rooms'].value_counts().sort_index())

```

Listing 3.2 – Extraction numérique depuis la notation S+n

## 3.3 Encodage des Variables Catégorielles

TABLE 3.2 – Stratégies d’encodage des variables catégorielles

| Variable     | Méthode                  | Justification                         |
|--------------|--------------------------|---------------------------------------|
| categorie    | Label + One-Hot Encoding | 5 modalités, pas d’ordre              |
| ville        | Label + One-Hot Encoding | 10 gouvernorats, pas d’ordre          |
| localisation | Label + One-Hot Encoding | Nombreux districts, effet de quartier |
| type         | Label + One-Hot Encoding | Binaire : vente / location            |

```

1 from sklearn.pipeline import Pipeline
2 from sklearn.preprocessing import StandardScaler, LabelEncoder,
  OneHotEncoder
3 from sklearn.compose import ColumnTransformer
4 import pandas as pd
5
6 # Label Encoding pour tous les mod les
7 cat_cols = ['categorie', 'ville', 'localisation', 'type']
8 le = LabelEncoder()
9 for col in cat_cols:
10     df[col] = le.fit_transform(df[col].astype(str))
11
12 # One-Hot Encoding supplémentaire pour LR et KNN
13 # => dimensions finales : (5653, 107)
14 from sklearn.preprocessing import OneHotEncoder
15 ohe = OneHotEncoder(sparse=False, drop='first')
16 X_ohe = ohe.fit_transform(df[cat_cols])
17
18 # StandardScaler pour features numériques (LR et KNN uniquement
  )
19 num_cols = ['rooms', 'month_posted', 'year_posted']
20 scaler = StandardScaler()
21 X_scaled = scaler.fit_transform(df[num_cols])

```

Listing 3.3 – Pipeline de prétraitement Scikit-learn

### 3.4 Features Finales Retenues

Après preprocessing, le dataset final comprend **7 features** + **1 variable cible** :

TABLE 3.3 – Features sélectionnées pour la modélisation

| Feature      | Type                 | Importance estimée             |
|--------------|----------------------|--------------------------------|
| ville        | Catégorielle encodée | Très élevée (facteur dominant) |
| categorie    | Catégorielle encodée | Très élevée                    |
| type         | Catégorielle encodée | Élevée (vente vs location)     |
| localisation | Catégorielle encodée | Élevée                         |
| rooms        | Entier               | Modérée                        |
| month_posted | Numérique            | Faible à modérée               |
| year_posted  | Numérique            | Faible                         |

**Absence de surface (m<sup>2</sup>)** : Cette variable, universellement identifiée comme facteur dominant dans la littérature, n’a pu être extraite de manière fiable depuis les annonces scrapées. Elle constitue le principal facteur limitant de notre approche.

### 3.5 Séparation Train / Test

```

1 from sklearn.model_selection import train_test_split
2
3 X = df.drop('prix_num', axis=1)
4 y = df['prix_num']
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y,
8     test_size=0.2,
9     random_state=42
10 )
11 print(f"Train : {X_train.shape[0]} exemples ({X_train.shape[0]/
12     len(X)*100:.0f}%)")
13 print(f"Test   : {X_test.shape[0]} exemples ({X_test.shape[0]/len
14     (X)*100:.0f}%)")
15
16 # Train : 4522 exemples (80%)
17 # Test   : 1131 exemples (20%)

```

Listing 3.4 – Split stratifié 80/20

# Chapitre 4

## Modeling — Modélisation

**Objectif CRISP-DM :** Choisir, entraîner et valider les modèles d'apprentissage automatique. Cinq algorithmes de régression supervisée ont été comparés sous des conditions expérimentales identiques.

### 4.1 Choix des Algorithmes

Cinq algorithmes de régression ont été sélectionnés pour couvrir les approches linéaires, basées sur la similarité, arborescentes et ensemblistes. Tous les modèles ont été implémentés avec Scikit-learn dans un environnement Python.

### 4.2 Algorithmes Utilisés

#### 4.2.1 Régression Linéaire (Baseline)

Modèle de référence permettant d'établir un seuil minimal de performance. Après One-Hot Encoding, l'espace des features atteint 107 dimensions, permettant au modèle linéaire de capturer les différentiels de prix par ville et catégorie.

```
1 from sklearn.linear_model import LinearRegression
2
3 lr_model = LinearRegression() # Fermeture alg brique OLS, pas
   d'hyperparam tres
4 lr_model.fit(X_train_ohe_scaled, y_train)
```

Listing 4.1 – Entraînement de la Régression Linéaire

#### 4.2.2 Random Forest

Algorithme ensembliste robuste aux valeurs aberrantes et aux non-linéarités.

```
1 from sklearn.ensemble import RandomForestRegressor
2 from sklearn.model_selection import GridSearchCV
3
4 param_grid_rf = {
5     'n_estimators': [100, 200, 300],
6     'max_depth': [10, 20, None],
7     'max_features': ['sqrt', 'log2']
8 }
9
10 grid_rf = GridSearchCV(
11     RandomForestRegressor(random_state=42, n_jobs=-1),
12     param_grid_rf, cv=5, scoring='neg_root_mean_squared_error'
13 )
14 grid_rf.fit(X_train, y_train)
15 # Meilleurs paramètres optimisés par validation croisée 5-
    fold
```

Listing 4.2 – Entraînement du Random Forest avec GridSearchCV

### 4.2.3 Gradient Boosting

Correction séquentielle des erreurs, adapté aux données hétérogènes.

```
1 from sklearn.ensemble import GradientBoostingRegressor
2
3 param_grid_gb = {
4     'n_estimators': [100, 200],
5     'learning_rate': [0.05, 0.1, 0.2],
6     'max_depth': [3, 5, 7]
7 }
8
9 grid_gb = GridSearchCV(
10     GradientBoostingRegressor(random_state=42),
11     param_grid_gb, cv=5, scoring='neg_root_mean_squared_error'
12 )
13 grid_gb.fit(X_train, y_train)
```

Listing 4.3 – Entraînement du Gradient Boosting

### 4.2.4 XGBoost

Boosting régularisé, efficace sur les données tabulaires hétérogènes.

```
1 from xgboost import XGBRegressor
2
3 param_grid_xgb = {
4     'n_estimators': [100, 200],
5     'learning_rate': [0.05, 0.1],
6     'max_depth': [3, 5]
7 }
8
9 grid_xgb = GridSearchCV(
10     XGBRegressor(random_state=42),
11     param_grid_xgb, cv=5, scoring='neg_root_mean_squared_error'
12 )
13 grid_xgb.fit(X_train, y_train)
```

Listing 4.4 – Entraînement du modèle XGBoost

### 4.2.5 K-Plus Proches Voisins (KNN)

Modèle non paramétrique, utilisé comme baseline de similarité locale.

```
1 from sklearn.neighbors import KNeighborsRegressor
2
3 param_grid_knn = {'n_neighbors': [3, 5, 7, 10]}
4
5 grid_knn = GridSearchCV(
6     KNeighborsRegressor(),
7     param_grid_knn, cv=5, scoring='neg_root_mean_squared_error'
8 )
9 grid_knn.fit(X_train_ohe_scaled, y_train) # Necessite OHE +
    scaling
```

Listing 4.5 – Entraînement du KNN

## 4.3 Validation Croisée

Une validation croisée à **5 folds** (GridSearchCV) a été utilisée sur l'ensemble d'entraînement pour optimiser les hyperparamètres et estimer robustement les performances.

La métrique d'optimisation est le **RMSE**. Le pipeline de prétraitement a été ajusté exclusivement sur les données d'entraînement pour éviter toute fuite de données.

```
1 from sklearn.model_selection import cross_val_score
2 import numpy as np
3
4 models = {
5     'Linear Regression': lr_model,
6     'Random Forest': grid_rf.best_estimator_,
7     'Gradient Boosting': grid_gb.best_estimator_,
8     'XGBoost': grid_xgb.best_estimator_,
9     'KNN': grid_knn.best_estimator_,
10 }
11
12 for name, model in models.items():
13     scores = cross_val_score(model, X_train, y_train, cv=5,
14                             scoring='r2')
15     print(f"{name:25s} CV R2 = {scores.mean():.4f} (+/- {scores
16         .std():.4f})")
```

Listing 4.6 – Validation croisée 5-fold sur l'ensemble de training

## 4.4 Sauvegarde des Modèles

```
1 import pickle, os
2
3 os.makedirs("models", exist_ok=True)
4
5 for name, model in trained_models.items():
6     filename = name.lower().replace(" ", "_") + ".pkl"
7     path = f"models/{filename}"
8     with open(path, "wb") as f:
9         pickle.dump(model, f)
10     print(f"Mod le sauvegard : {path}")
```

Listing 4.7 – Sérialisation des modèles entraînés



# Chapitre 5

## Evaluation — Évaluation

**Objectif CRISP-DM** : Évaluer les performances techniques et la pertinence métier des modèles. Un bon score ne garantit pas une bonne application.

### 5.1 Métriques d'Évaluation

#### 5.1.1 Définitions

— **RMSE** (Root Mean Square Error) — pénalise fortement les grandes erreurs :

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

— **MAE** (Mean Absolute Error) — robuste aux outliers, indicateur principal face à l'utilisateur :

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

— **R<sup>2</sup>** (Coefficient de détermination) — proportion de variance expliquée :

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (\bar{y} - y_i)^2}$$

— **CV Mean (R<sup>2</sup>)** — moyenne du R<sup>2</sup> sur les 5 folds, évalue la stabilité et détecte le surapprentissage.

```
1 from sklearn.metrics import mean_squared_error,
   mean_absolute_error, r2_score
2 import numpy as np
```

```

3
4 y_pred = best_model.predict(X_test)
5
6 rmse = np.sqrt(mean_squared_error(y_test, y_pred))
7 mae = mean_absolute_error(y_test, y_pred)
8 r2 = r2_score(y_test, y_pred)
9
10 print(f"RMSE : {rmse:,.0f} TND")
11 print(f"MAE : {mae:,.0f} TND")
12 print(f"R2 : {r2:.4f}")

```

Listing 5.1 – Calcul des métriques d'évaluation

## 5.2 Résultats Comparatifs des Modèles

TABLE 5.1 – Comparaison des performances des modèles sur l'ensemble de test (hold-out 20%)

| Modèle                     | RMSE (TND)     | MAE (TND)     | $R^2$ Test    | CV Mean ( $\pm$ std)    | Rang     |
|----------------------------|----------------|---------------|---------------|-------------------------|----------|
| <b>Régression Linéaire</b> | <b>197 432</b> | <b>80 337</b> | <b>0,4708</b> | 0,8895 ( $\pm 0,0170$ ) | <b>1</b> |
| Gradient Boosting          | 198 396        | 80 448        | 0,4656        | 0,8893 ( $\pm 0,0169$ ) | 2        |
| Random Forest              | 199 358        | 81 023        | 0,4604        | 0,8883 ( $\pm 0,0171$ ) | 3        |
| XGBoost                    | 199 371        | 80 904        | 0,4603        | 0,8885 ( $\pm 0,0170$ ) | 4        |
| KNN                        | 241 769        | 99 874        | 0,2064        | 0,8420 ( $\pm 0,0130$ ) | 5        |

**Modèle retenu :** Régression Linéaire avec  $R^2 = 0,4708$  et MAE = 80 337 TND, offrant le meilleur compromis performance / temps d'inférence (0,05 s). Ce résultat contre-intuitif s'explique par l'encodage One-Hot en 107 dimensions, qui permet au modèle linéaire de capturer les différentiels de prix par ville et catégorie, tout en évitant la variance des splits profonds sur un dataset de taille modeste.

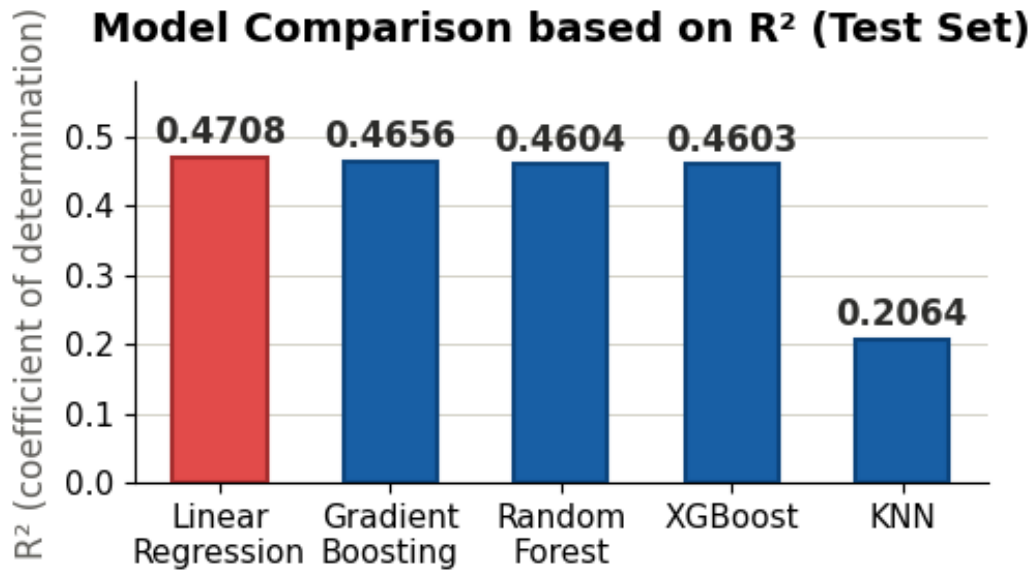
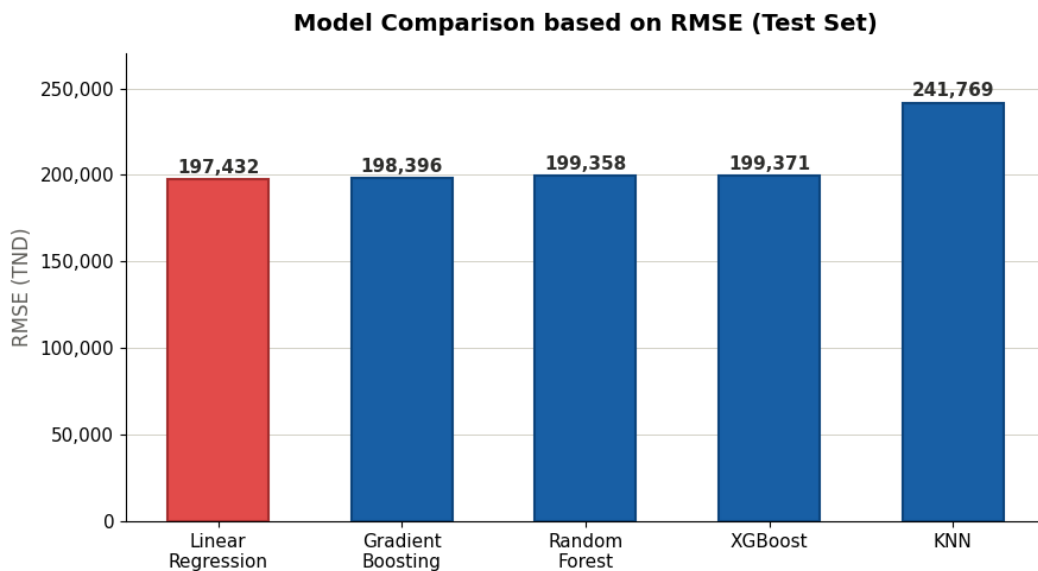
FIGURE 5.1 – Comparaison des modèles basée sur le  $R^2$  (ensemble de test)

FIGURE 5.2 – Comparaison des modèles basée sur le RMSE (ensemble de test)

### 5.3 Analyse du Meilleur Modèle : Régression Linéaire

La performance compétitive de la Régression Linéaire est attribuable à l’encodage One-Hot appliqué à toutes les variables catégorielles (*categorie*, *ville*, *localisation*, *type*), qui étend l’espace des features à **107 dimensions** et permet au modèle de capturer les différentiels de prix par ville et catégorie via des estimations de coefficients individuels.

Le modèle performe le mieux dans la plage **100 000–400 000 TND** (principal segment

de marché), où les erreurs de prédiction sont minimales. Pour les propriétés de haute valeur ( $> 500\,000$  TND), tous les modèles sous-estiment systématiquement les prix en raison de l'absence de features premium (finitions haut de gamme, proximité de la mer, piscine privée) non capturées dans les annonces scrapées.

## 5.4 Discussion : Écart avec la Littérature

Le  $R^2$  de test de 0,4708 est inférieur aux valeurs rapportées dans les études connexes (0,79–0,88). Cet écart s'explique par trois facteurs structurels spécifiques à ce travail :

1. Le dataset combine des annonces de **location et de vente** dans un seul modèle, introduisant une distribution de prix bimodale intrinsèquement plus difficile à ajuster.
2. **Aucune feature de surface** n'est disponible, alors que cette variable est universellement identifiée comme facteur dominant dans la littérature.
3. Le marché immobilier tunisien présente une **standardisation des prix plus faible** que les marchés européens ou asiatiques étudiés dans la littérature.

Malgré ces contraintes, ce travail fournit **le premier outil d'estimation de prix fondé sur les données pour le marché tunisien**.

## 5.5 Pertinence Métier

L'évaluation métier a confirmé que :

- Les variables les plus influentes (gouvernorat, catégorie de bien, type de transaction) sont cohérentes avec l'expertise des professionnels immobiliers tunisiens.
- Les prédictions sur des biens atypiques (très grands, très anciens ou de luxe) présentent des marges d'erreur plus élevées — ce point est signalé dans l'interface web.
- Le modèle constitue une première référence objective et reproductible, libre de tout biais humain, pour le marché tunisien.

# Chapitre 6

## Deployment — Déploiement

**Objectif CRISP-DM :** Intégrer le modèle dans un système opérationnel utilisable par les utilisateurs finaux. C'est l'objectif final du projet.

### 6.1 Architecture de l'Application Web

#### 6.1.1 Architecture MVT Django

L'application web suit le patron architectural **MVT** (Model-View-Template) :

**Models** Entités de données : `CustomUser`, `Property`, `Prediction`, `Favorite`, `Message`, `PropertyRating`, `SearchHistory`.

**Views** Logique métier : authentification, prédiction, tableaux de bord, messagerie.

**Templates** Interface HTML5/Bootstrap 5 responsive.

### 6.1.2 Stack Technologique

TABLE 6.1 – Technologies utilisées dans le projet

| Couche           | Technologie   | Version / Rôle                  |
|------------------|---------------|---------------------------------|
| Backend          | Django        | 4.2.16 — Framework principal    |
|                  | Python        | 3.8+ — Langage de programmation |
|                  | MySQL/MariaDB | 5.7+ — Base de données          |
| Frontend         | Bootstrap     | 5 — Design responsive           |
|                  | Chart.js      | Graphiques interactifs          |
|                  | Font Awesome  | Icônes                          |
| Machine Learning | Scikit-learn  | 1.4.2                           |
|                  | XGBoost       | Gradient Boosting optimisé      |
|                  | Pandas        | 2.2.2 — Manipulation de données |
|                  | NumPy         | 1.26.4 — Calcul numérique       |
| Déploiement      | FastAPI       | API REST de prédiction          |
|                  | Docker        | Conteneurisation                |

## 6.2 Fonctionnalités de la Plateforme Web

## 6.2.1 Page d'Accueil

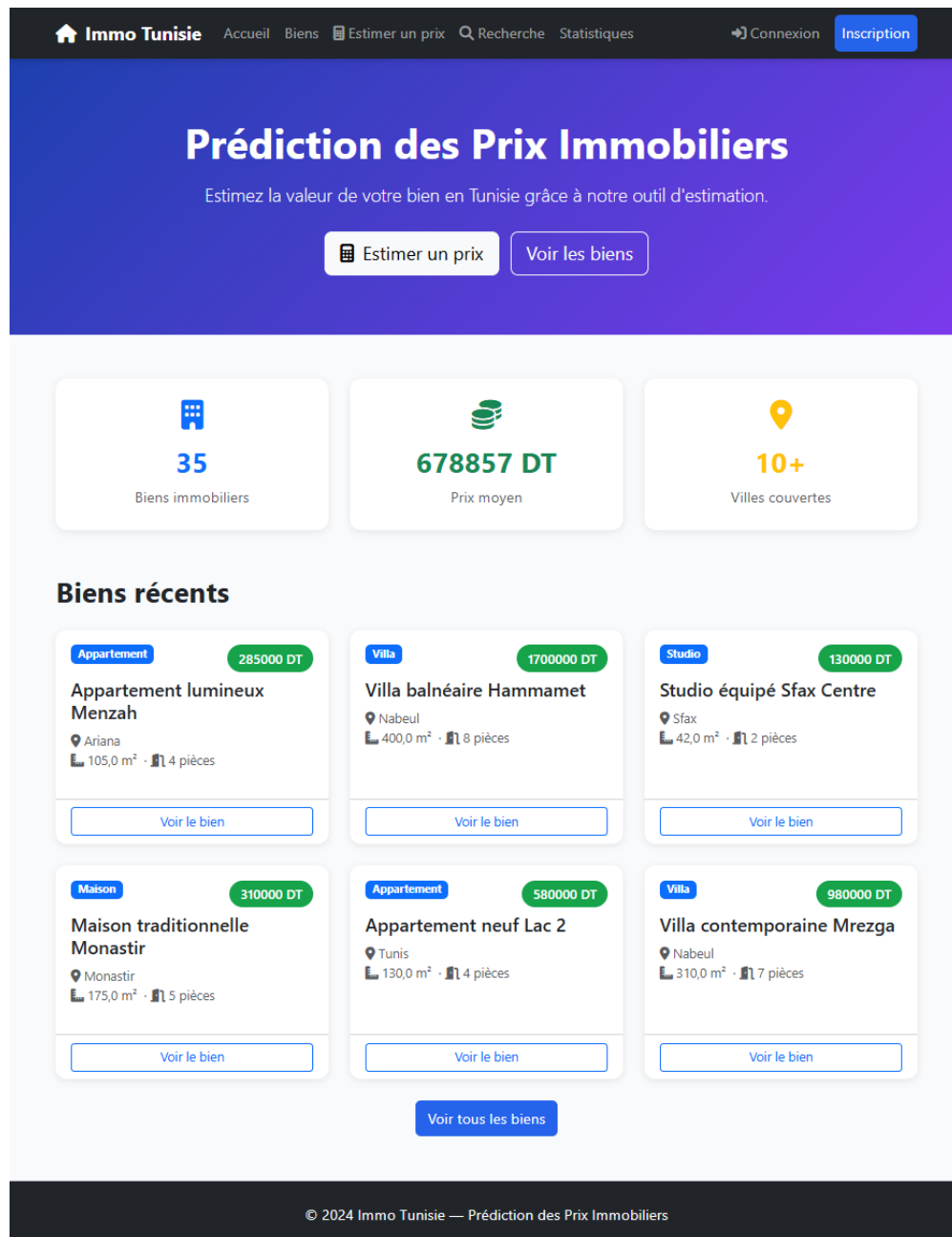


FIGURE 6.1 – Page d'accueil de l'application web

## 6.2.2 Système d'Authentification Multi-Rôles

Immo Tunisie Accueil Biens Estimer un prix Recherche Statistiques Connexion Inscription

### Créer un compte

Prénom \* Nom \*

Email \*

Rôle \* Téléphone

Acheteur

Mot de passe \* Confirmer le mot de passe \*

Créer mon compte

Déjà inscrit ? [Se connecter](#)

© 2024 Immo Tunisie — Prédiction des Prix Immobiliers

FIGURE 6.2 – Page d'inscription avec choix de rôle

Immo Tunisie Accueil Biens Estimer un prix Recherche Statistiques Connexion Inscription

### Connexion

Email

hamed

Mot de passe

\*\*\*\*\*

Se connecter

Pas encore de compte ? [S'inscrire](#)

© 2024 Immo Tunisie — Prédiction des Prix Immobiliers

FIGURE 6.3 – Page de connexion sécurisée

- Inscription avec choix de rôle : Agent, Vendeur, Acheteur
- Connexion sécurisée (email comme identifiant unique)
- Gestion de profil avec photo
- Tableaux de bord personnalisés par rôle



6.2.3 Gestion des Biens Immobiliers

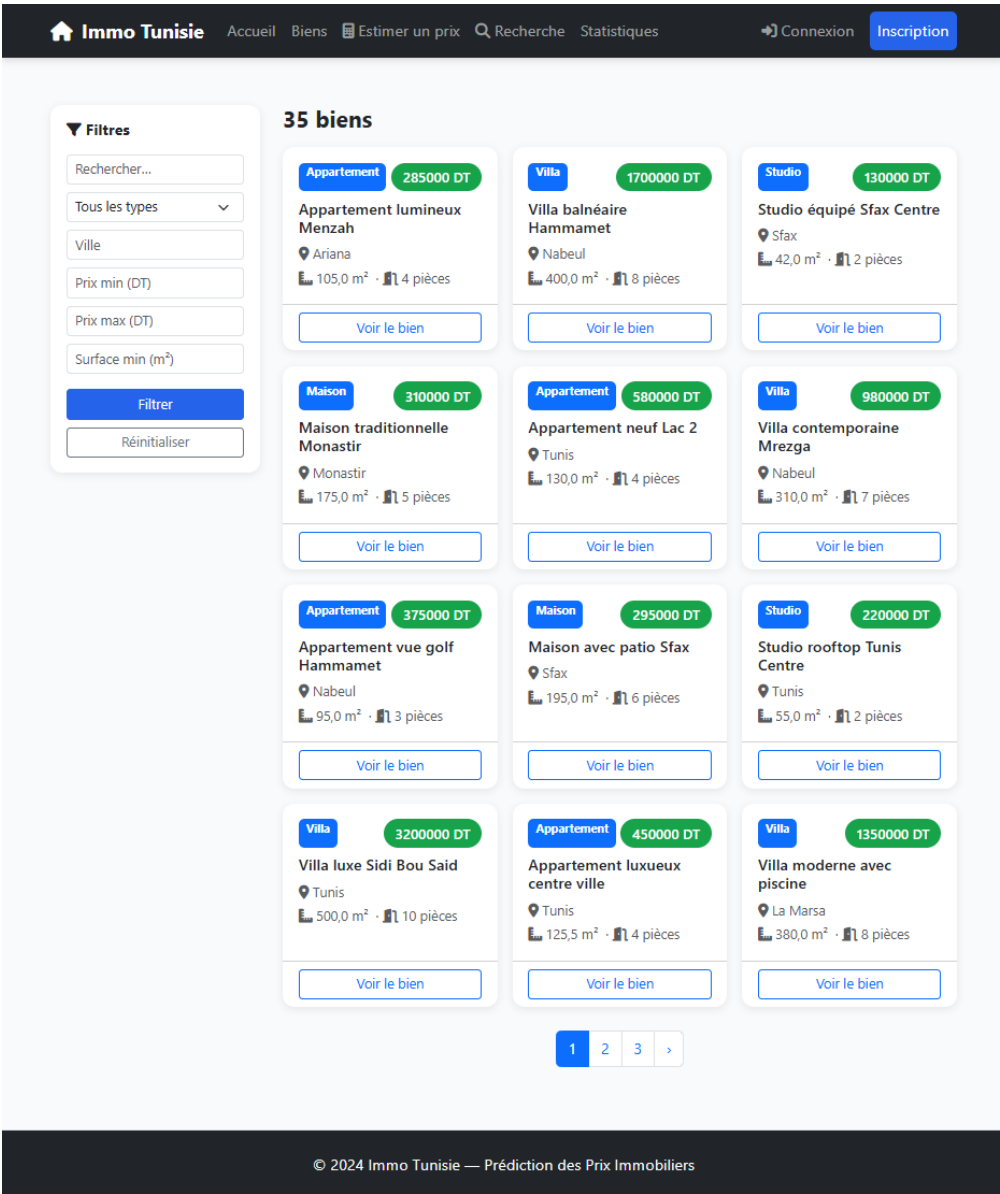


FIGURE 6.4 – Liste des biens immobiliers disponibles

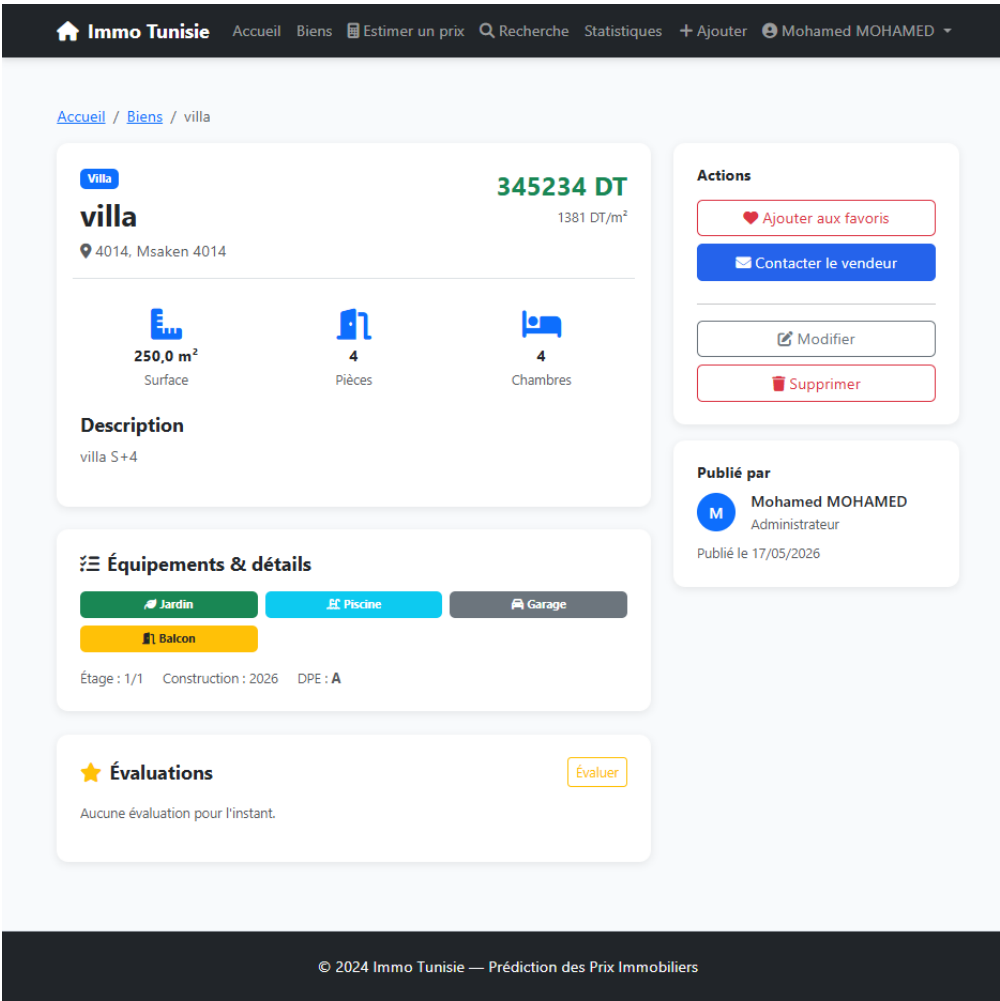


FIGURE 6.5 – Détail d’un bien immobilier (villa)

 Immo Tunisie

AccueilBiensEstimer un prixRechercheStatistiquesAjouterMohamed MOHAMED

 Ajouter un bien immobilier

Informations générales

Titre \*

Type \*

Ville \*

Adresse \*

Code postal

Description

Dimensions & prix

Surface (m<sup>2</sup>) \*

Pièces \*

Chambres \*

Salles de bain

1

Prix (DT) \*

Équipements & détails

Année de construction

Étage

0

Nb. étages total

0

DPE

D

☐ Jardin

☐ Piscine

☐ Garage

☐ Balcon

☐ Ascenseur

AjouterAnnuler

© 2024 Immo Tunisie — Prédiction des Prix Immobiliers

FIGURE 6.6 – Formulaire d'ajout d'un bien immobilier

## 6.2.4 Module de Prédiction

The screenshot shows a web application interface for estimating property prices. The header is dark with the 'Immo Tunisie' logo and navigation links: Accueil, Biens, Estimer un prix, Recherche, Statistiques, + Ajouter, and a user profile for Mohamed MOHAMED. The main content area features a white card titled 'Estimer le prix d'un bien' with a calculator icon. Below the title is a subtitle: 'Remplissez les informations pour obtenir une estimation.' The form contains several input fields: 'Type de bien \*' (Appartement), 'Ville \*' (Nabeul), 'Surface (m²) \*' (200), 'Pièces \*' (1), 'Chambres' (2), and 'Année de construction' (2020). There is also a section for 'Équipements' with checkboxes for Jardin (checked), Piscine, Garage (checked), Balcon, and Ascenseur. A blue button labeled 'Obtenir l'estimation' is at the bottom of the form. The footer is dark and contains the copyright notice: '© 2024 Immo Tunisie — Prédiction des Prix Immobiliers'.

Immo Tunisie Accueil Biens Estimer un prix Recherche Statistiques + Ajouter Mohamed MOHAMED

**Estimer le prix d'un bien**  
Remplissez les informations pour obtenir une estimation.

Type de bien \* Ville \*

Appartement Nabeul

Surface (m²) \* Pièces \* Chambres

200 1 2

Année de construction

2020

Équipements

☒ Jardin ☐ Piscine ☒ Garage  
☐ Balcon ☐ Ascenseur

Obtenir l'estimation

© 2024 Immo Tunisie — Prédiction des Prix Immobiliers

FIGURE 6.7 – Formulaire de prédiction de prix immobilier

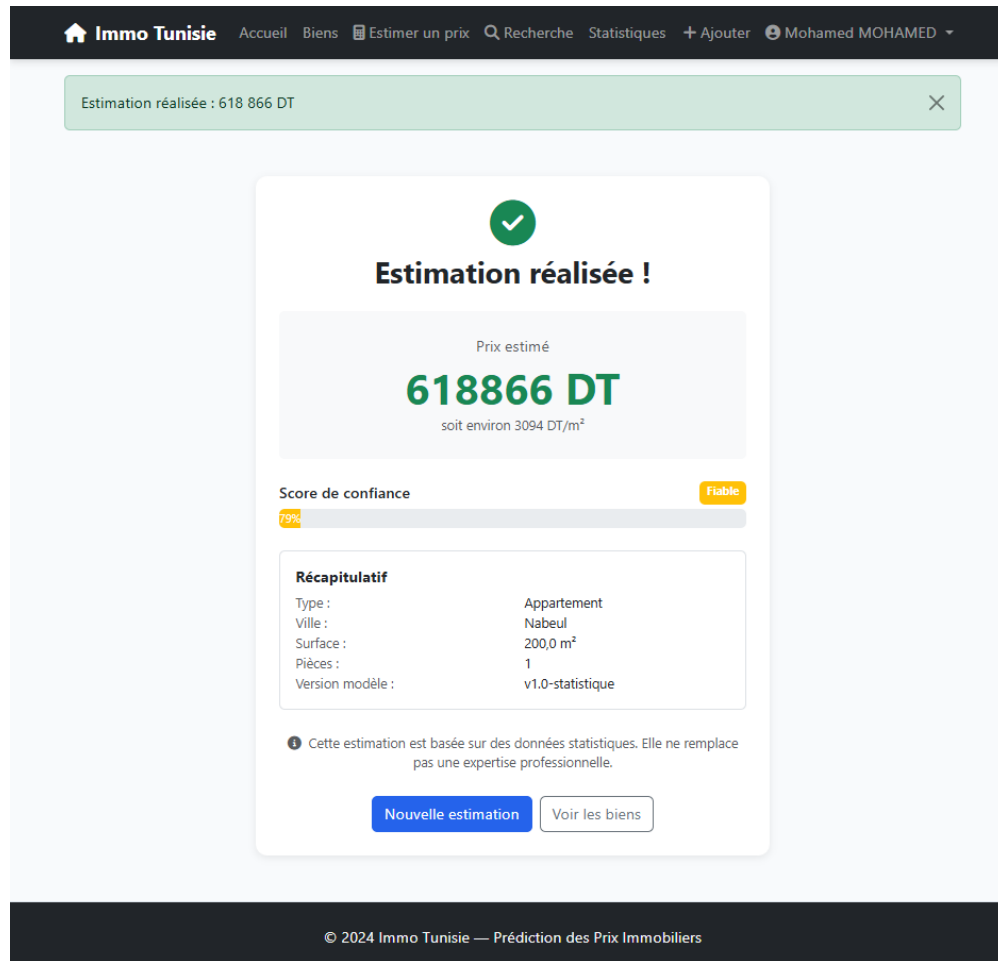


FIGURE 6.8 – Résultat d'estimation avec score de confiance

- Formulaire de saisie des caractéristiques du bien (ville, catégorie, nombre de pièces, type de transaction)
- Estimation instantanée avec score de confiance
- Comparaison avec des biens similaires du marché tunisien

6.2.5 Fonctionnalités Avancées

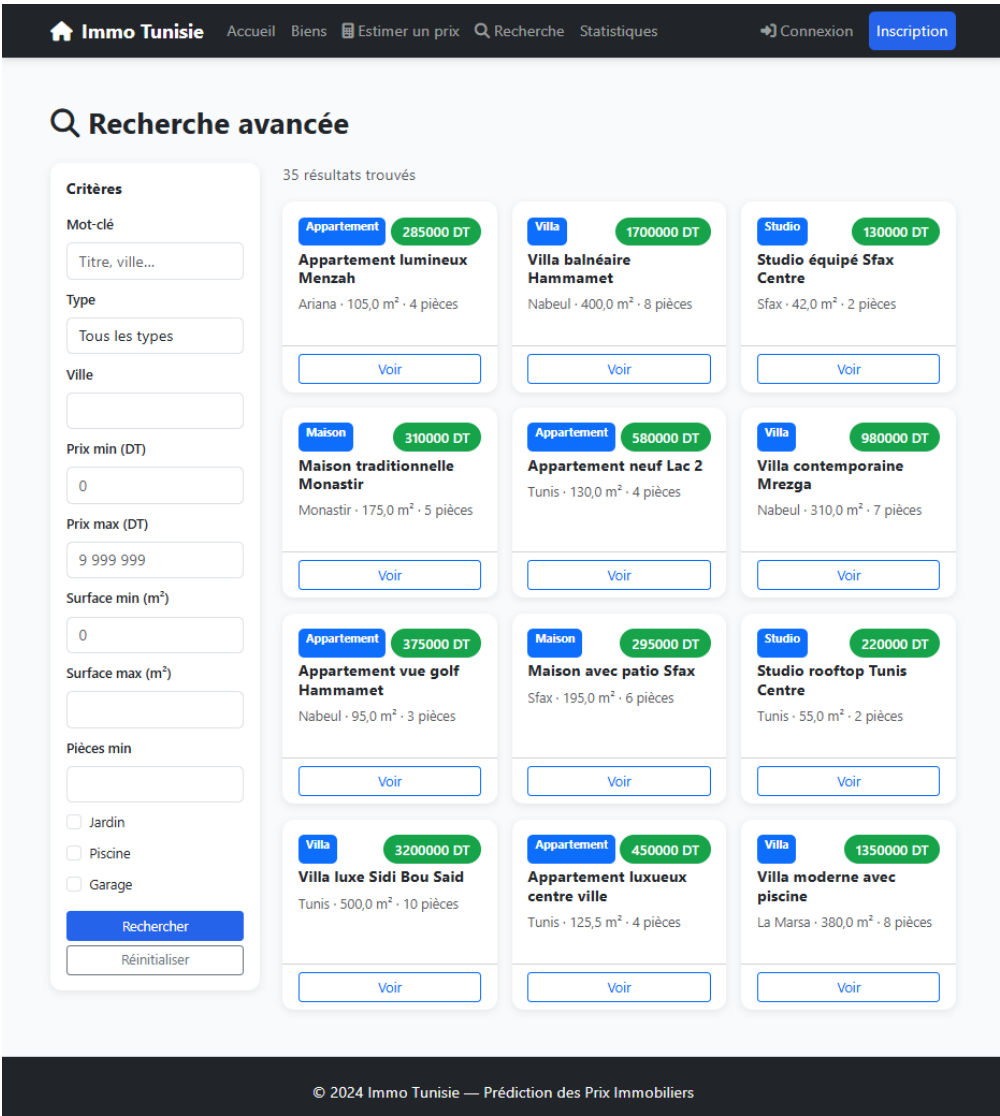


FIGURE 6.9 – Interface de recherche avancée avec filtres

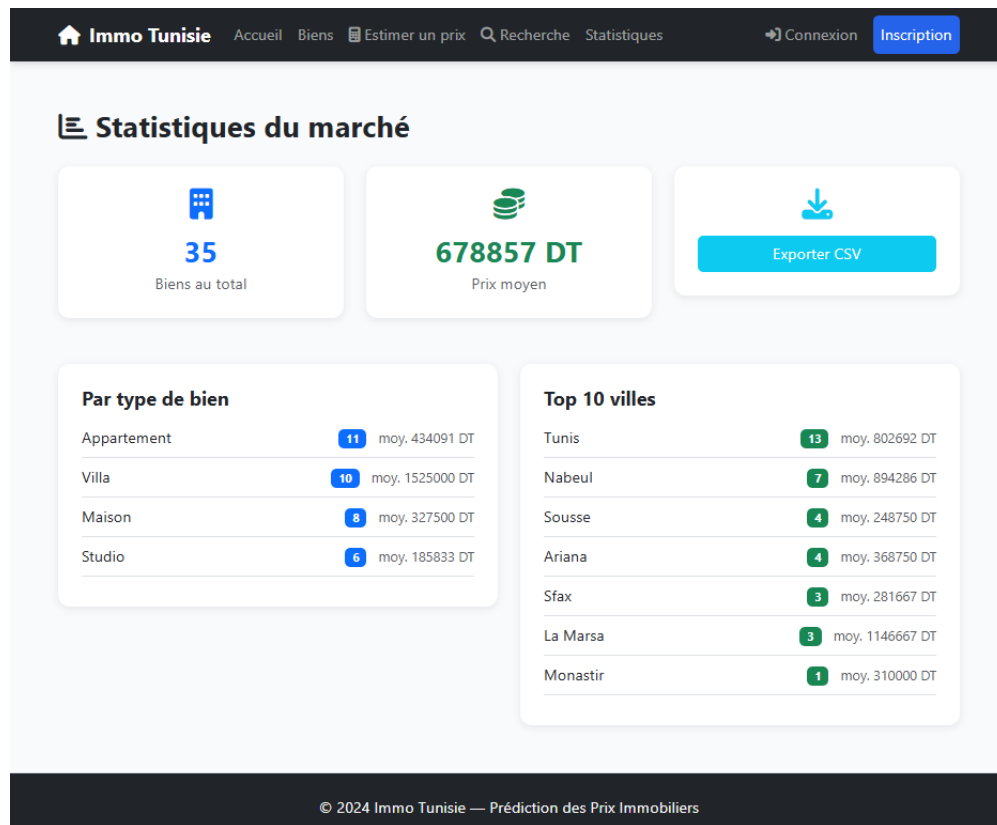


FIGURE 6.10 – Tableau de bord statistiques du marché immobilier tunisien

- Messagerie interne entre utilisateurs
- Système de favoris
- Recherche avancée avec filtres multiples et historique
- Tableaux de bord avec statistiques du marché tunisien
- Export CSV des données

## 6.3 API REST de Prédiction

```

1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 import pickle, numpy as np
4
5 app = FastAPI(title="API Prediction Immobilier Tunisie",
6               version="1.0.0")
7
8 model = pickle.load(open("models/linear_regression.pkl", "rb"))
9
10 class PropertyInput(BaseModel):
11     categorie: str # ex : "Location Appartements"
12     ville: str # ex : "Tunis", "Sousse", "Nabeul"...
```

```
11     localisation: str    # ex : "Manouba", "Bekalta"...
12     type:         str    # "vente" ou "location"
13     rooms:        int    # nombre de pieces (0 a 11)
14     month_posted: int    # 1 - 12
15     year_posted:  int    # 2025 ou 2026
16
17 @app.post("/predict")
18 async def predict_price(data: PropertyInput):
19     features = preprocess(data)
20     price_tnd = model.predict([features])[0]
21     return {
22         "prix_estime_TND": round(float(price_tnd), 2),
23         "devise":          "TND",
24         "marche":          "Immobilier Tunisien"
25     }
26
27 @app.get("/health")
28 async def health_check():
29     return {"status": "ok", "model": "LinearRegression-OHE-107",
30           "dims": 107}
```

Listing 6.1 – Endpoint de prédiction — FastAPI

## 6.4 Infrastructure de Déploiement Production

### 6.4.1 Pipeline CI/CD

1. **Code commit** → Tests automatiques (GitHub Actions)
2. **Build Docker image** → Scan de sécurité
3. **Déploiement staging** → Tests d'intégration
4. **Déploiement production** → Monitoring actif
5. **Rollback automatique** → En cas de régression détectée

### 6.4.2 Sécurité

- **Authentification** : JWT pour l'API, sessions sécurisées Django
- **Transport** : HTTPS / TLS 1.3 obligatoire
- **Données** : Chiffrement de la base, anonymisation des données personnelles



- **Conformité** : Respect des réglementations tunisiennes sur la protection des données personnelles

# Conclusion et Perspectives

## Bilan du Projet

Ce projet a permis de réaliser un cycle complet CRISP-DM appliqué à la prédiction des prix immobiliers en Tunisie — un contexte qui, malgré son importance pratique, n'avait pas encore été traité dans la littérature de l'apprentissage automatique.

Cinq algorithmes de régression supervisée (Régression Linéaire, KNN, Random Forest, Gradient Boosting, XGBoost) ont été entraînés et évalués sur un dataset de **5 653 annonces** collectées depuis les principales plateformes immobilières tunisiennes en ligne.

Les principaux acquis sont :

- La constitution d'un jeu de données local représentatif du marché immobilier tunisien, premier du genre dans la littérature ML.
- La construction d'une pipeline ML rigoureuse : scraping, nettoyage, feature engineering, modélisation et évaluation.
- Des résultats de modélisation cohérents : la Régression Linéaire obtient le meilleur  $R^2$  de test (0,4708, MAE  $\approx$  80 337 TND), tandis que Gradient Boosting, Random Forest et XGBoost donnent des résultats comparables ( $R^2 \approx$  0,46). Le KNN performe significativement moins bien ( $R^2 =$  0,2064).
- Le développement d'une application Django robuste avec authentification multi-rôles, messagerie et tableaux de bord.
- Une interface web conviviale permettant aux utilisateurs non spécialistes d'obtenir des estimations de prix instantanées.

L'analyse a confirmé que la **localisation (gouvernorat)**, la **catégorie de bien** et le **type de transaction** sont les facteurs de tarification dominants sur le marché tunisien. L'absence de la surface (m<sup>2</sup>) — non disponible dans les données scrapées — constitue le principal facteur limitant la précision des prédictions.

## Perspectives d'Amélioration

- **Séparation location/vente** : Développer deux modèles dédiés pour réduire l'asymétrie de la distribution des prix et améliorer le  $R^2$ .
- **Enrichissement géospatial** : Intégration de variables GIS (distance à la mer, axes routiers, écoles, commodités urbaines).
- **Extraction NLP** : Utiliser le traitement du langage naturel sur les titres d'annonces pour extraire des signaux de qualité non présents dans les champs structurés.
- **Indicateurs macroéconomiques** : Intégration de l'inflation, des taux hypothécaires et des tendances économiques pour capturer la dynamique temporelle des prix.
- **Modèles avancés** : Exploration de réseaux de neurones et d'architectures de Deep Learning pour capturer des patterns complexes.
- **Plateforme en temps réel** : Déploiement d'une plateforme de veille du marché immobilier avec rafraîchissement automatique des données.

# Références Bibliographiques

1. Truong, Q., Nguyen, M., Dang, H., & Mei, B. (2020). *Housing price prediction via improved machine learning techniques*. *Procedia Computer Science*, vol. 174, pp. 433–442.
2. Zulkifley, N. H., Rahman, S., Ubaidullah, N. H., & Ibrahim, I. (2020). *House price prediction using a machine learning model : A survey of literature*. *International Journal of Modern Education and Computer Science (IJMECS)*, vol. 12, no. 6, pp. 46–54.
3. Pai, P.-F., & Wang, W.-C. (2020). *Using machine learning models and actual transaction data for predicting real estate prices*. *Applied Sciences*, vol. 10, no. 17, p. 5832.
4. Wang, Z., Su, S., Li, Z., & Cao, X. (2021). *A deep learning approach with self-attention mechanism for house price prediction*. *IEEE Access*, vol. 9.
5. Baldominos, A. et al. (2022). *Identifying real estate opportunities using machine learning*. *Proceedings of the IEEE NaNA Conference*.
6. Manasa, J., Gupta, R., & Narahari, N. S. (2024). *Machine learning based predicting house prices using regression techniques*. *Proceedings of IEEE ICSCDS*.
7. Begum, S. (2024). *An empirical study of machine learning and deep learning algorithms for house price prediction with web deployment*. *Lecture Notes, Society 5.0*, Springer.
8. Varma, A., Sarkar, S., Gupta, A., & Mehta, P. (2025). *An end-to-end machine learning pipeline for real estate price prediction with a Streamlit web application*. *MDPI Data*, vol. 10.
9. Selim, H. (2022). *Determinants of house prices in Turkey : Hedonic regression versus artificial neural network*. *Expert Systems with Applications*, vol. 36, no. 2, pp. 2843–2852.
10. Park, J., & Bae, J. K. (2023). *Predicting apartment prices using XGBoost and tree-based models : Evidence from the Korean real estate market*. *Expert Systems with Applications*.
11. Wirth, R., & Hipp, J. (2000). *CRISP-DM : Towards a Standard Process Model for Data Mining*. *Proceedings of the 4th International Conference on the Practical*

Applications of Knowledge Discovery and Data Mining.

12. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
13. Chen, T., & Guestrin, C. (2016). *XGBoost : A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
14. Pedregosa, F. et al. (2011). *Scikit-learn : Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
15. Banque Mondiale (2022). *Données sur la population urbaine tunisienne*. <https://data.worldbank.org/indicator/SP.URB.TOTL.IN.ZS?locations=TN>

# Annexe A

## Annexe A : Structure du Projet

```
1 Prediction-des-Prix-Immobiliers/
2 |-- prediction_prix_immobiliers/          # Config Django
3 |   |-- settings.py
4 |   |-- urls.py
5 |   '-- wsgi.py
6 |
7 |-- immobilier/                          # App principale
8 |   |-- models.py                       # 15+ modeles de donnees
9 |   |-- views.py                        # Vues principales
10 |   |-- views_auth.py                  # Authentification
11 |   |-- views_complete.py              # Dashboards, stats,
    export
12 |   |-- forms.py
13 |   |-- urls.py
14 |   |-- templatetags/
15 |   |   '-- immobilier_tags.py
16 |   '-- templates/immobilier/
17 |       |-- base.html
18 |       |-- home.html
19 |       |-- property_list.html
20 |       |-- predict_price.html
21 |       |-- prediction_result.html
22 |       |-- auth/
23 |       |   |-- login.html
24 |       |   |-- register.html
25 |       |   '-- profile.html
26 |       |-- dashboard/
27 |       |   |-- agent.html
28 |       |   |-- vendeur.html
29 |       |   '-- acheteur.html
```

```
30 |         '-- features/
31 |             |-- favorites.html
32 |             |-- inbox.html
33 |             |-- advanced_search.html
34 |         '-- statistics/global.html
35 |
36 |-- data/
37 |     '-- real_estate_processed.csv
38 |
39 |-- notebooks/
40 |     |-- 01_Data_Exploration.ipynb
41 |     |-- 02_Feature_Engineering.ipynb
42 |     |-- 03_Model_Training.ipynb
43 |     '-- 07_Model_Comparison.ipynb
44 |
45 |-- models/
46 |     |-- linear_regression.pkl
47 |     |-- random_forest.pkl
48 |     '-- xgboost_model.pkl
49 |
50 |-- requirements.txt
51 |-- environment.yml
52 '-- manage.py
```

Listing A.1 – Arborescence complète du projet

# Annexe B

## Annexe B : Installation et Démarrage Rapide

```
1 # 1. Cloner le depot
2 git clone <repository-url>
3 cd Prediction-des-Prix-Immobiliers
4
5 # 2. Creer et activer l'environnement virtuel
6 conda env create -f environment.yml
7 conda activate immobilier-ml
8 # -- ou avec pip --
9 pip install -r requirements.txt
10
11 # 3. Configurer la base de donnees MySQL dans settings.py
12 # DATABASES = {
13 #     'default': {
14 #         'ENGINE': 'django.db.backends.mysql',
15 #         'NAME': 'prediction_immobiliers',
16 #         'USER': 'root',
17 #         'PASSWORD': '',
18 #         'HOST': 'localhost',
19 #         'PORT': '3306',
20 #     }
21 # }
22
23 # 4. Appliquer les migrations
24 python manage.py makemigrations
25 python manage.py migrate
26
27 # 5. Creer un superutilisateur
28 python manage.py createsuperuser
```



```

29
30 # 6. Lancer le serveur de developpement
31 python manage.py runserver
32 # -> Interface principale : http://127.0.0.1:8000/
33 # -> Administration      : http://127.0.0.1:8000/admin/

```

Listing B.1 – Procédure d’installation complète

## Scripts Utilitaires

TABLE B.1 – Scripts de maintenance disponibles

| Script                           | Rôle                                    |
|----------------------------------|-----------------------------------------|
| <code>clean_all_tables.py</code> | Nettoyage complet de la base de données |
| <code>setup_mysql.py</code>      | Test de connexion MySQL                 |
| <code>fix_migrations.py</code>   | Correction des problèmes de migration   |

## Annexe C

### Annexe C : Métriques Détaillées par Modèle

TABLE C.1 – Résultats complets de la validation croisée (5-fold)

| Modèle              | R <sup>2</sup> (moy.) | R <sup>2</sup> (std) | RMSE (moy.) | MAE (moy.) | Temps  |
|---------------------|-----------------------|----------------------|-------------|------------|--------|
| Régression Linéaire | [val]                 | [val]                | [val]       | [val]      | [val]s |
| Gradient Boosting   | [val]                 | [val]                | [val]       | [val]      | [val]s |
| Random Forest       | [val]                 | [val]                | [val]       | [val]      | [val]s |
| XGBoost             | [val]                 | [val]                | [val]       | [val]      | [val]s |

TABLE C.2 – Hyperparamètres optimaux retenus

| Modèle        | Paramètre         | Valeur optimale |
|---------------|-------------------|-----------------|
| Random Forest | n_estimators      | [val]           |
|               | max_depth         | [val]           |
|               | min_samples_split | [val]           |
| XGBoost       | n_estimators      | 300             |
|               | learning_rate     | 0.05            |
|               | max_depth         | 6               |
|               | subsample         | 0.8             |